

CLASSIFICATION SYSTEM

NLP-BASED RESUME-JOB MATCHING

Presented by Carla Radieddine

PROJECT OVERVIEW

- Recruitment often requires manual screening of large numbers of resumes.
- This process is time-consuming, subjective, and difficult to scale.
- Natural Language Processing (NLP) can help automate the analysis of resume and job-description text.
- The main goal is to identify whether a candidate is a good match for a specific job based on textual information.

PROBLEM STATEMENT

The project focuses on resume-job matching classification.

Input:

- Resume-related text
- Job-related text

Output:

- Match
- No Match

The original dataset contains a matched score.

This score was converted into a binary class using a threshold of 0.7

- Match (≥ 0.7)
- No match (< 0.7)

DATASET OVERVIEW

- Source: Resume Dataset for Job Matching (Kaggle)
- Number of records: 9,544
- Each record represents a resume-job pair
- Includes: Resume fields, Job requirement fields and Matched score

- Features: 35
- Resume fields: skills, education, positions, responsibilities
- Job fields: title, required skills, responsibilities, education requirements

DATASET FEATURES

The model learns from both the candidate profile and the job description.

Resume-related features:	Job-related features:
Career objective	Job position name
Skills	Skills required
Degree names	Responsibilities
Major field of study	Educational Requirements
Company names	Experience Requirements
Positions	-----
Responsibilities	-----

EXPLORATORY DATA ANALYSIS

- 01 The analysis focused on:
- score distribution
 - class balance
 - text length
 - vocabulary size
 - frequent tokens and bigrams
 - noise detection
-

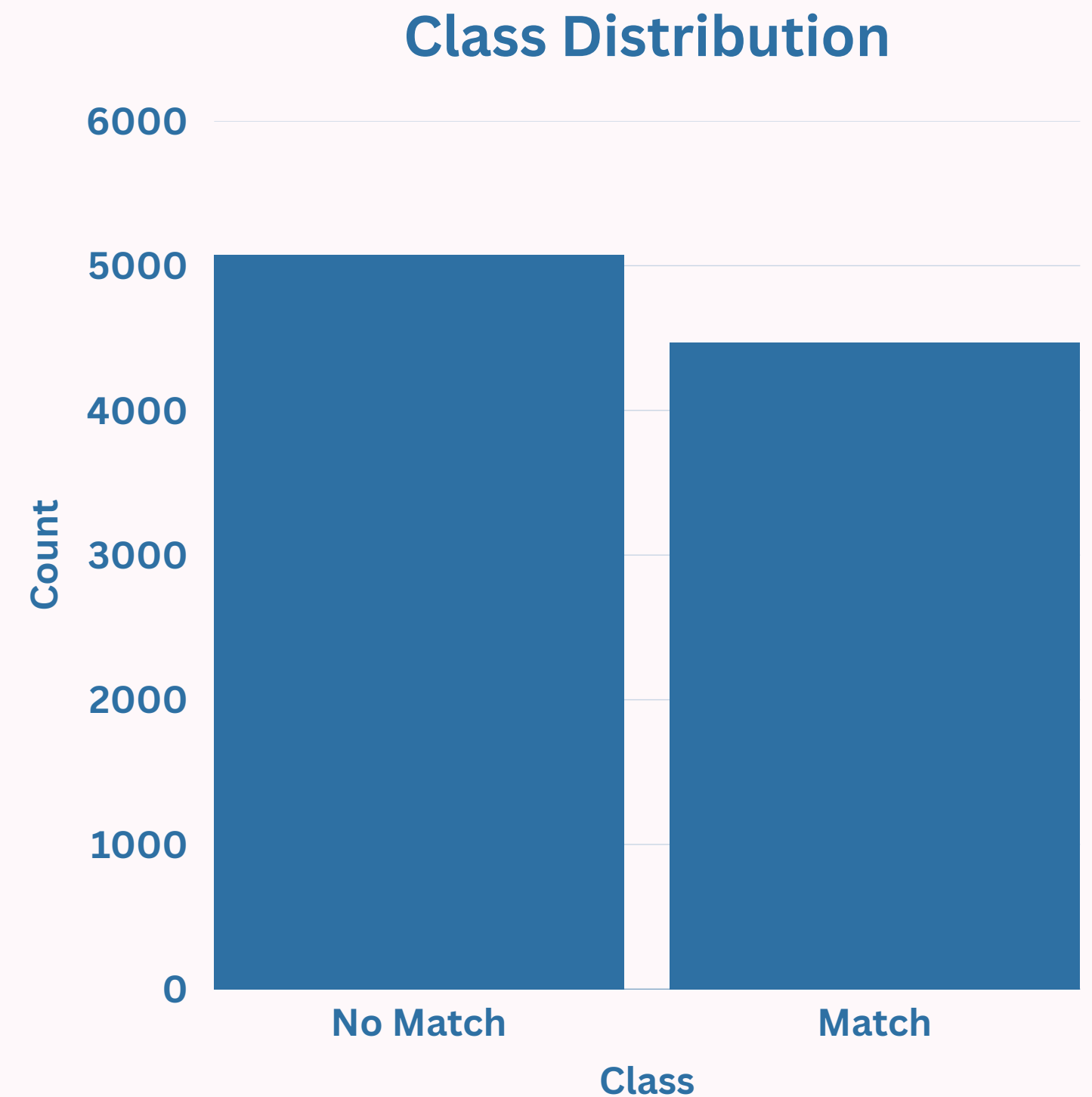
- 02 Purpose:
- understand data quality
 - identify patterns
 - detect imbalance and redundancy

EDA: Key Findings

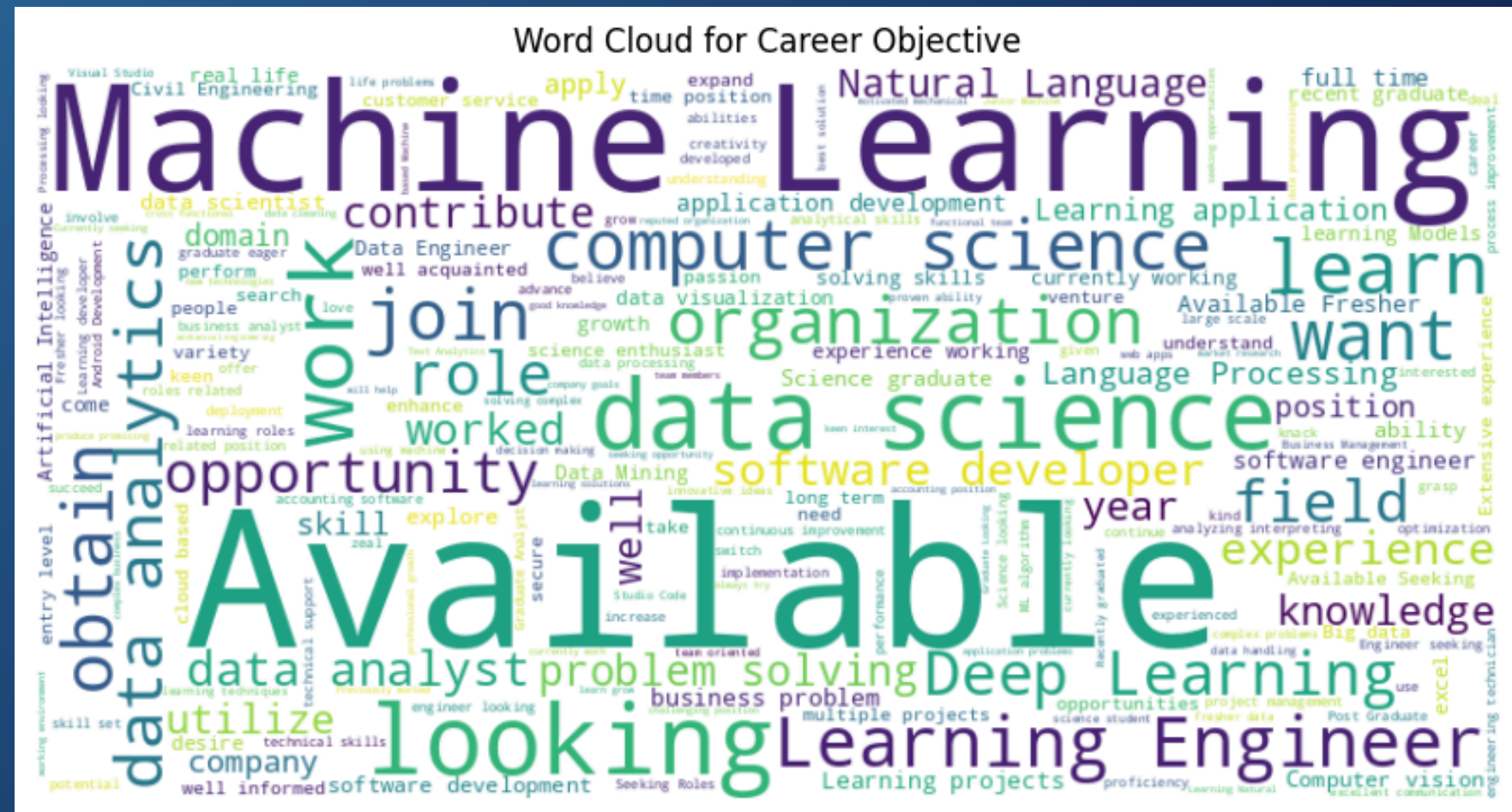
1	Class distribution is relatively balanced: No Match (53.17%) and Match (46.83%)
2	Vocabulary size: 2385 tokens
3	Average combined text length: ~104 words
4	Common frequent terms: "management", "development", "analysis", "data", "science"
5	Noise detected in tokens like: "name", "year", "available"

Class Distribution

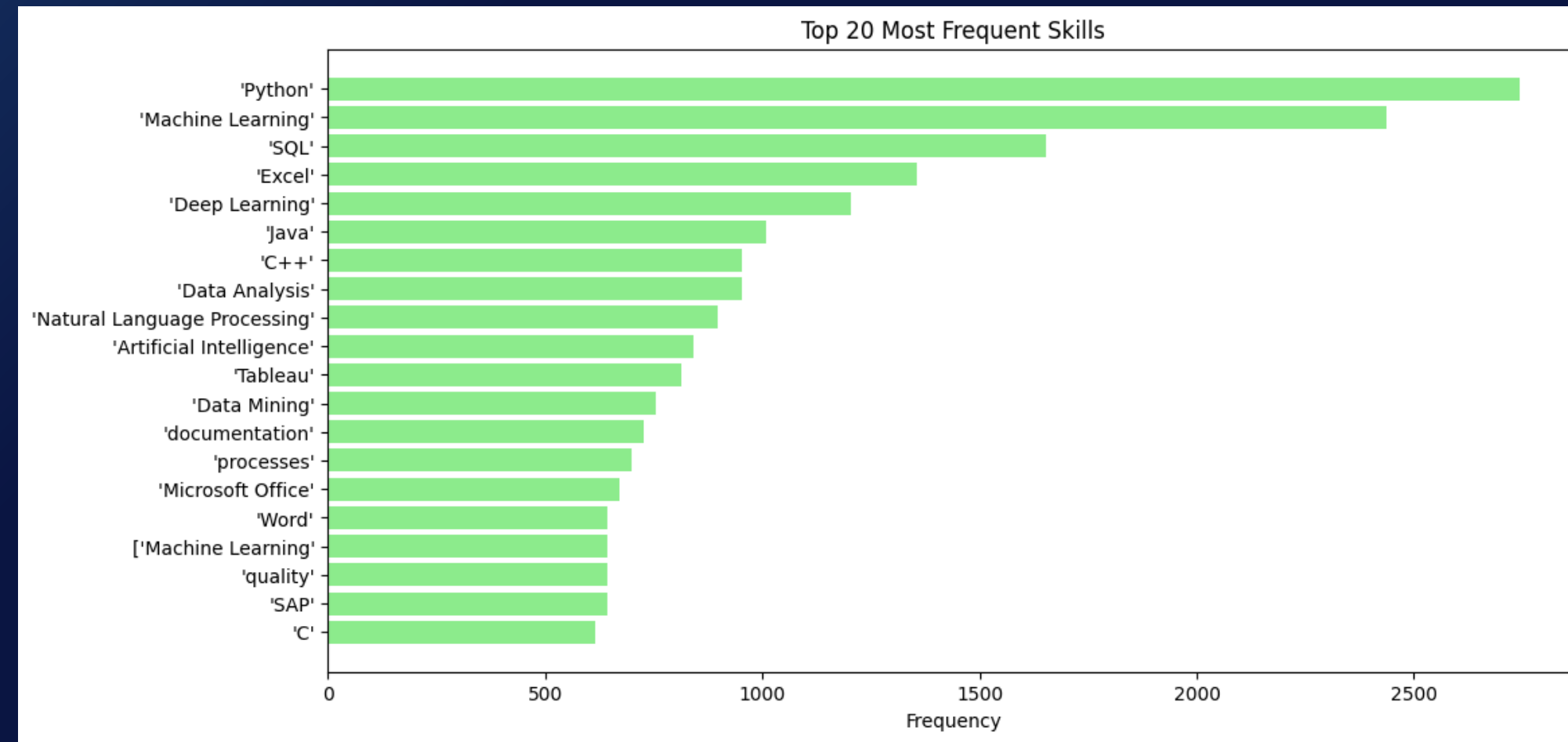
The class distribution confirmed a mild imbalance but no severe skew, supporting the use of standard classification models.



EDA Visual Insights



Word cloud showed frequent career terms such as: "machine learning", "data science", "engineer".



Top skills included: "Python", "Machine Learning", "SQL", "Excel", "Deep Learning".

Text Preprocessing

01

Missing text values were filled using a placeholder

03

Irrelevant symbols and noisy text were reduced

02

Text was cleaned and normalized using: lowercasing, tokenization, stopwords removal, lemmatization.

04

Selected fields were merged into a single feature: combined_text

Feature Engineering and Text Representation

Two text representations were used:

01. TF-IDF

- Used for Logistic Regression and Random Forest
- Captures importance of words in documents
- Effective for keyword-driven tasks

02. Tokenized Sequences

- Used for LSTM
- Text converted into integer sequences
- Sequences padded to fixed length

Models Implemented

Machine Learning Models

- Logistic Regression
- Random Forest

Reason for comparison

- Compare traditional NLP methods with deep learning
- Evaluate which approach performs best on this task

Deep Learning Model

LSTM

Training Strategy

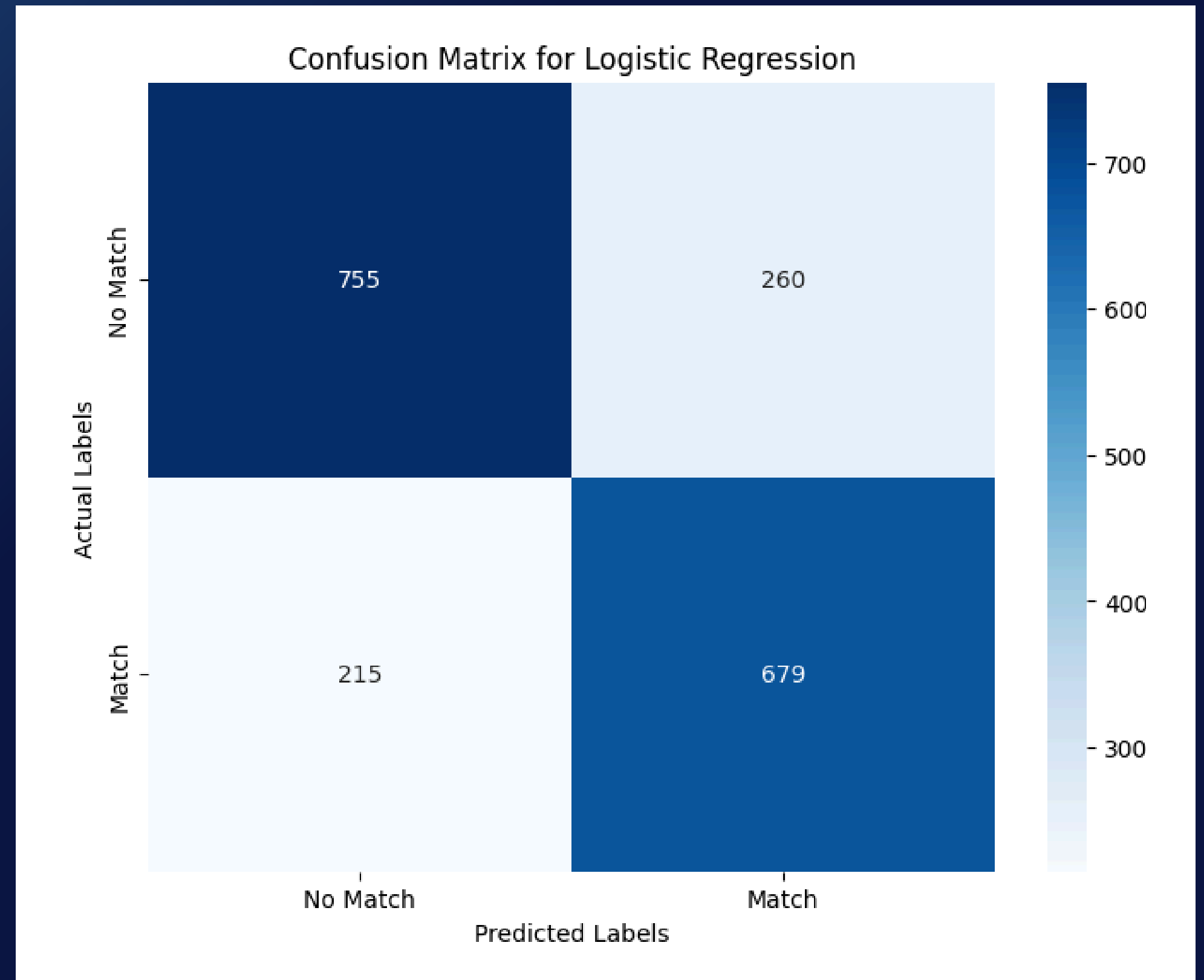
- 01** Dataset split: 80% training and 20% testing
- 02** Stratified sampling used to preserve class balance
- 03** Threshold 0.7 applied to generate binary labels
- 04** LSTM training used: Adam optimizer, binary cross-entropy loss, and early stopping to reduce overfitting
- 05** Random Forest tuning used
RandomizedSearchCV

Logistic Regression Results

- Accuracy: 0.75
- Precision: 0.72
- Recall: 0.76
- F1-score: 0.74

Interpretation:

- Strong baseline performance
- Balanced precision and recall
- Effective on sparse TF-IDF features

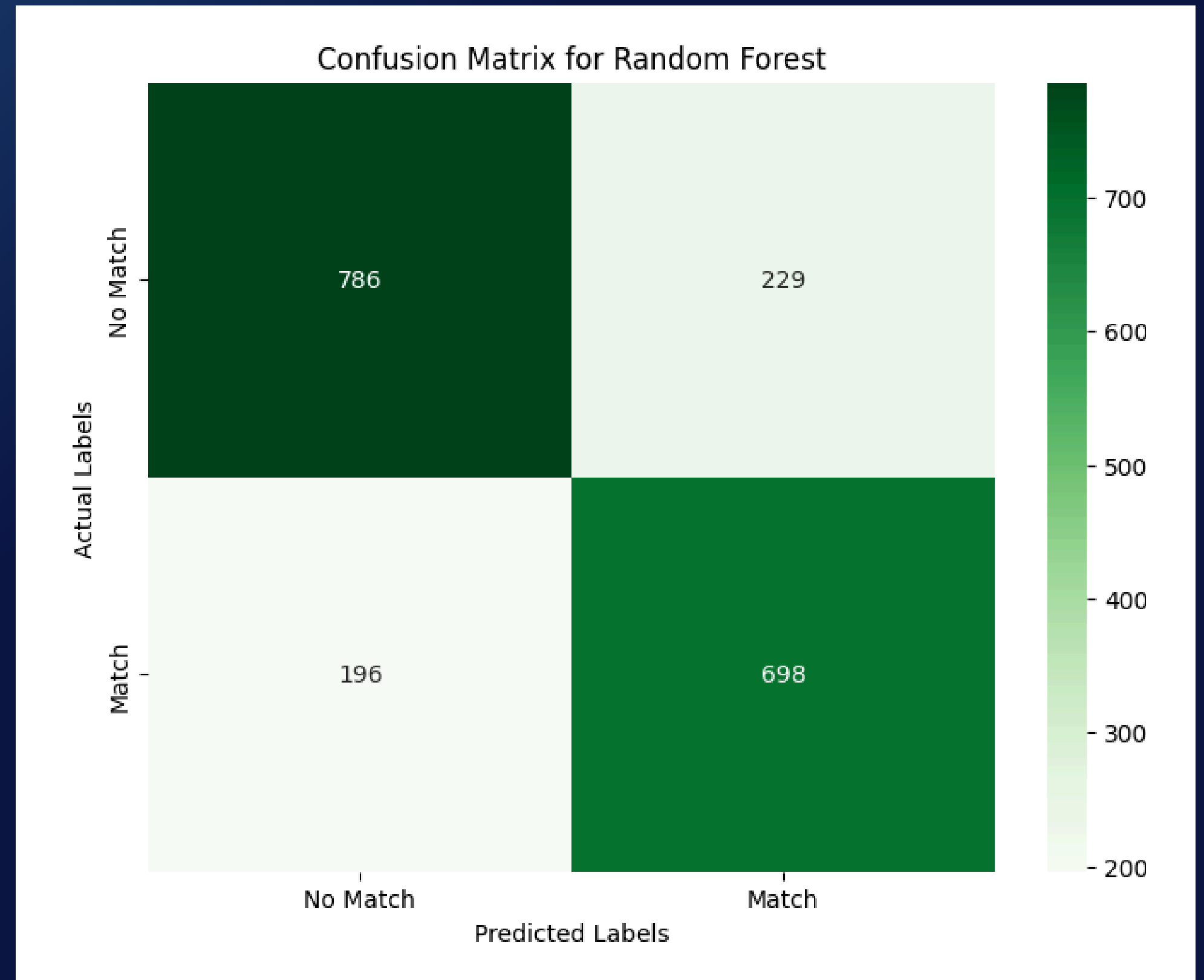


Random Forest Results

- Accuracy: 0.78
- Precision: 0.75
- Recall: 0.78
- F1-score: 0.77

Interpretation:

- Best overall model
- Strong class separation
- More effective than Logistic Regression
- Handles noisy, high-dimensional features well

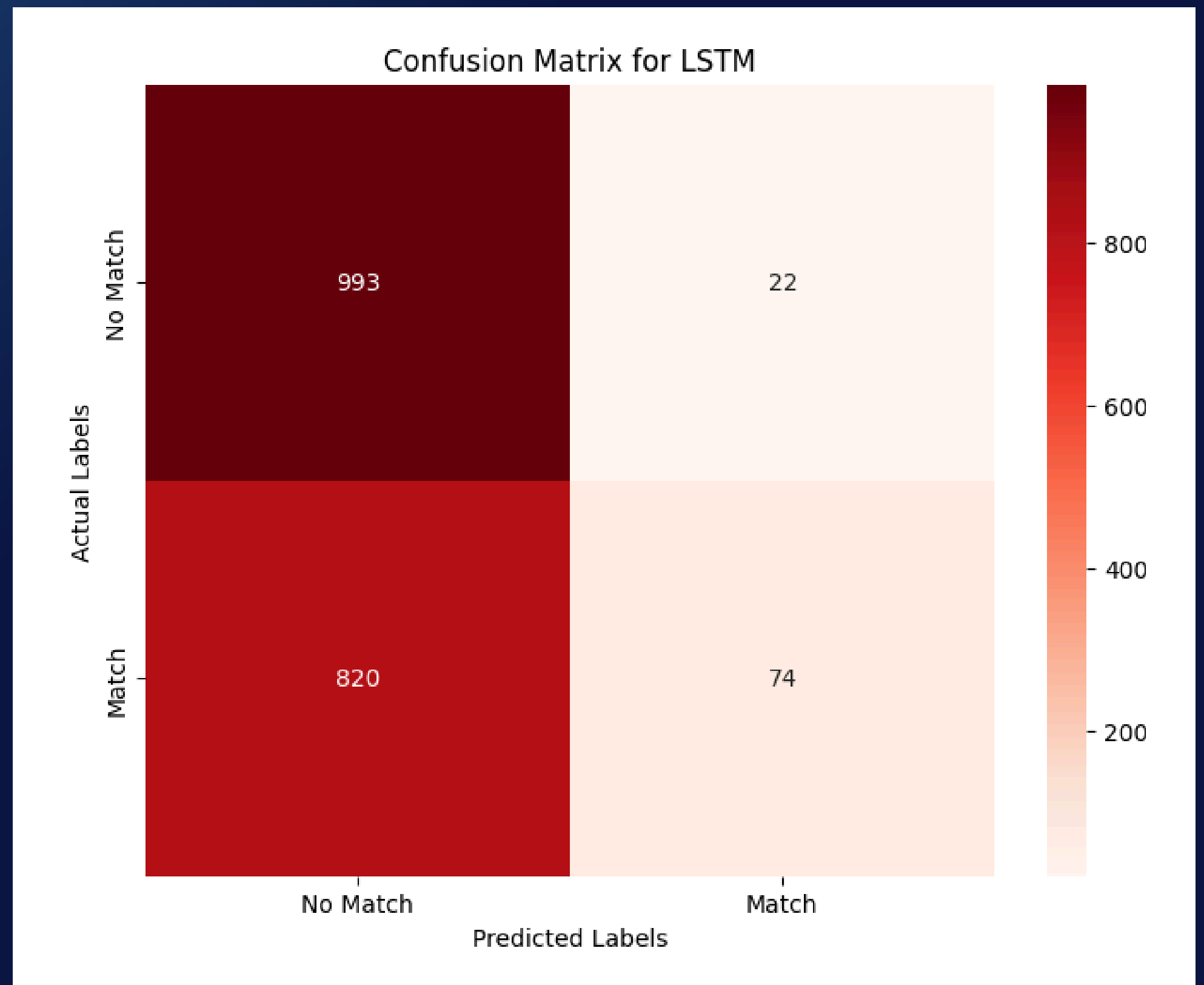


LSTM Results

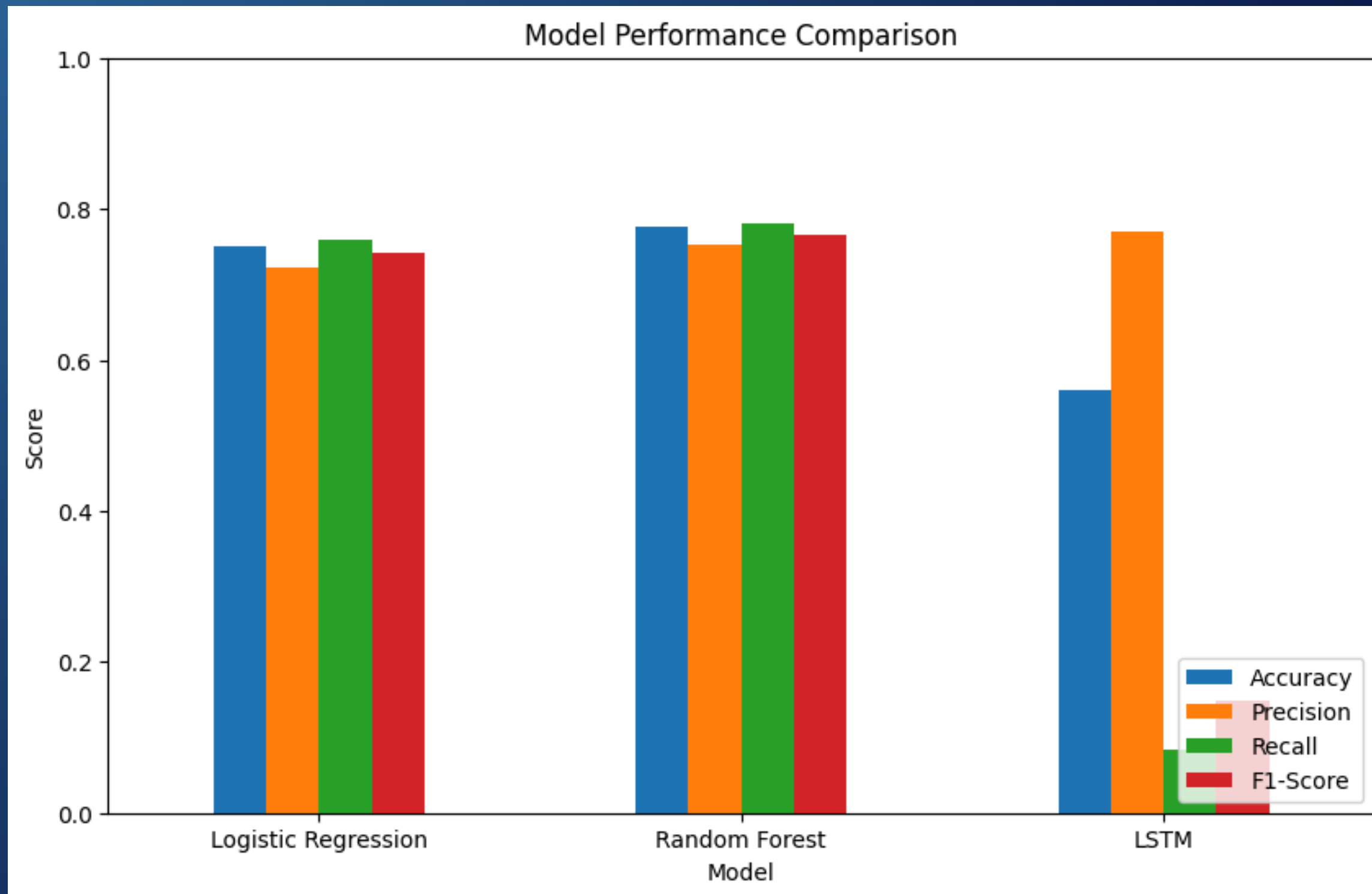
- Accuracy: 0.56
- Precision: 0.77
- Recall: 0.08
- F1-score: 0.15

Interpretation:

- Very poor recall
- Predicted very few positive matches
- Biased toward “No Match”
- Underperformed compared to classical models



Model Comparison



- Random Forest achieved the highest accuracy and F1-score
- Logistic Regression performed well with balanced results
- LSTM failed to detect most true matches (low recall)
- TF-IDF models proved more effective for this task

Error Analysis

False Negatives	False Positives
Missed true matches	Incorrect matches
Weak or unclear keywords	Common overlapping terms
Lack of strong skills	Frequent terms
Vague descriptions	Keyword over-reliance

Key Insight:

- Model relies heavily on keyword overlap
- Struggles with weak or ambiguous text

Hyperparameter Optimization

Applied to Random Forest using
RandomizedSearchCV

Tuned parameters:

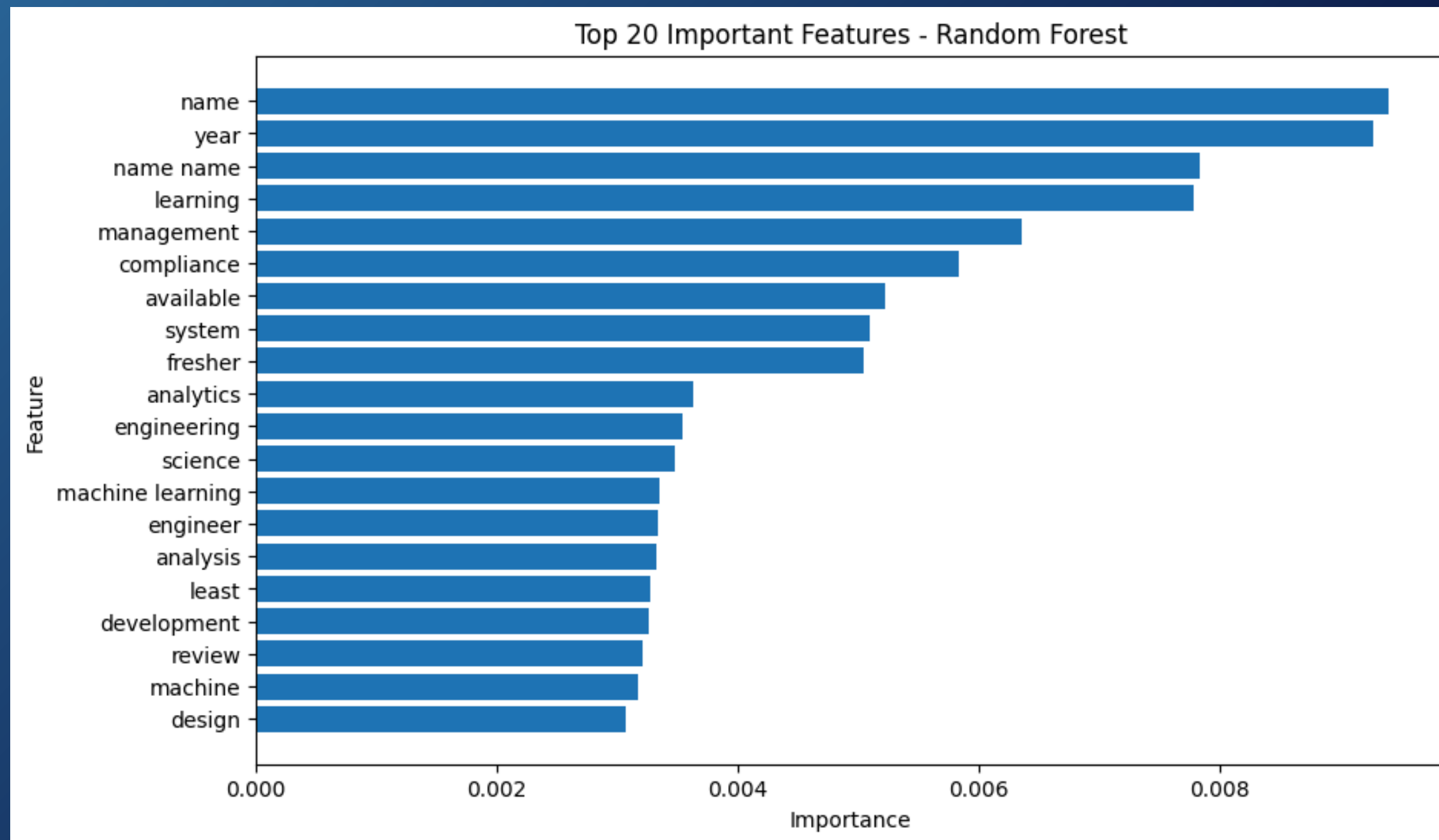
- `n_estimators = 150`
- `max_depth = 20`
- `min_samples_split = 2`
- `min_samples_leaf = 1`

Best cross-validation F1-score: 0.7688

Tuned test performance was slightly
lower than the original model

Original Random Forest was retained
as final model

Model Interpretability

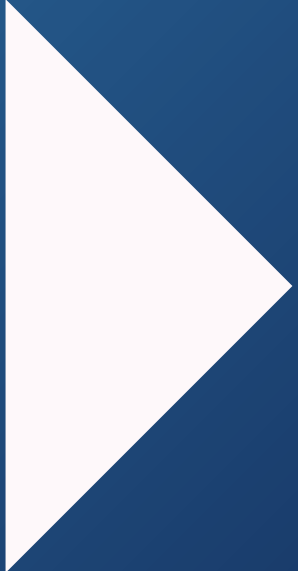


Feature importance from Random Forest showed important terms such as: “learning”, “management”, “analytics”, “engineering”, “machine learning”, “development”.

Interpretation:

- Model relies on meaningful technical keywords
- Some noisy terms still appear: “name”, “year”.

Ethical Considerations

- 
1. Dataset is biased toward technical roles
 2. Keyword-based matching may disadvantage candidates who use different terminology
 3. Noisy or incomplete data may affect fairness
 4. The system should be used as: a decision-support tool and not a fully automated hiring system

Reproducibility

- ✓ Fixed random seeds were used
- ✓ Consistent preprocessing pipeline applied
- ✓ Same train/test split maintained
- ✓ Main tools: pandas, scikit-learn, NLTK, TensorFlow/Keras

Conclusion

Built an end-to-end NLP pipeline for resume-job matching

Random Forest achieved the best performance

TF-IDF was highly effective for this keyword-driven task

LSTM underperformed on this dataset

Classical models outperformed deep learning in this case

THANK YOU!